1. Importações e Inicialização

```python
from flask import Flask, render_template, request
```

`from flask import ...`: Importa recursos específicos da biblioteca Flask instalada no ambiente Python.

`Flask`: A classe principal que cria e configura a aplicação web.

`render_template`: Função que carrega arquivos HTML (normalmente localizados na pasta `templates/`) e substitui marcações dinâmicas (Jinja2) pelos dados enviados pelo Python.

`request`: Objeto global do Flask que intercepta e disponibiliza todas as informações enviadas pelo navegador na requisição atual (como parâmetros de URL ou dados de formulário).

```python
aplicacao = Flask(__name__)
```

Instancia o objeto central da aplicação.

`__name__` é uma variável interna do Python que indica o nome do módulo atual. O Flask utiliza esse valor para identificar onde estão as pastas de templates e arquivos estáticos (CSS, imagens, JS).

2. Dados em Memória (Simulação de Banco de Dados)

```python
POSTAGENS = [
    {
        "titulo": "Primeiros Passos na Lógica com Python",
           "resumo": "Entenda como variáveis, listas e funções transformam ideias em programas reais.",
        "categoria": "Python",
```

```
        "autor": "Prof. de Tecnologia",
        "data": "10 Mai 2026",
                                                    "imagem":
"https://images.unsplash.com/photo-1526379095098-d400fd0bf935?auto=format&fit=crop&w=600&q=80"
    },
    ...
]
```

POSTAGENS: Uma lista (`[...]`) global de dicionários (`{...}`).

Cada dicionário representa um artigo/post com propriedades bem definidas (`titulo`, `resumo`, `categoria`, `autor`, `data`, `imagem`). Como está em memória RAM, esses dados não sofrem alterações diretas pelo usuário final e servem como catálogo do site.

Python
```
COMENTARIOS = [
    {
        "nome": "Maria Souza",
         "mensagem": "Muito fácil de entender a separação entre o HTML e o Python!"
    }
]
```

COMENTARIOS: Outra lista de dicionários que armazena os comentários postados pelos visitantes. Novos comentários enviados pelo formulário serão inseridos aqui dinamicamente.

3. Função de Filtragem (`filtrar_postagens`)

```python
def filtrar_postagens(lista_completa, palavra_buscada, categoria_escolhida):
```

Declara uma função auxiliar que recebe três parâmetros:

- `lista_completa`: A lista com todos os posts (`POSTAGENS`).
- `palavra_buscada`: O termo textual que o usuário digitou na busca.
- `categoria_escolhida`: A categoria selecionada no filtro (ou "Todas").

```python
    resultado = []
```

Cria uma lista vazia que receberá apenas os itens que cumprirem os dois critérios de filtro.

```python
    for post in lista_completa:
```

Inicia uma estrutura de repetição que itera sobre cada post da lista original.

```python
        passou_na_categoria = (
            categoria_escolhida == "Todas" or
                                   post["categoria"].lower() == categoria_escolhida.lower()
        )
```

Avalia a correspondência da categoria:

- Retorna `True` se a categoria selecionada for `"Todas"`.
- Ou retorna `True` se a categoria do post atual for idêntica à selecionada.

O método **`.lower()`** converte ambos os lados para letras minúsculas, garantindo uma comparação que ignora maiúsculas e minúsculas (*case-insensitive*).

```python
                    texto_do_post = (post["titulo"] + " " + post["resumo"]).lower()

                    passou_na_busca = palavra_buscada.lower() in texto_do_post
```

`texto_do_post`: Concatena o título e o resumo do post em uma única string em minúsculas.

`passou_na_busca`: Avalia com o operador `in` se o texto digitado pelo usuário está presente dentro desse bloco textual unificado. Se `palavra_buscada` for vazia `""`, o operador `in` avalia automaticamente como `True`.

```python
        if passou_na_categoria and passou_na_busca:

            resultado.append(post)
```

Se o post satisfizer simultaneamente o filtro de categoria e de busca textual, ele é inserido no final da lista `resultado` via **`.append()`**.

```python
    return resultado
```

Finaliza a função devolvendo a lista contendo apenas os posts filtrados.

4. Rota Principal (`/`)

```python
@aplicacao.route("/", methods=["GET"])

def pagina_inicial():
```

`@aplicacao.route("/", methods=["GET"])`: Decorador que mapeia a URL raiz do site (`/`) ao método HTTP `GET` (o método padrão ao carregar uma página no navegador).

`def pagina_inicial():`: Função associada (*view function*) disparada sempre que o endereço `/` é acessado.

```
Python
    termo_busca = request.args.get("busca", "").strip()

            categoria_atual   =   request.args.get("categoria",
"Todas").strip()
```

`request.args.get(chave, valor_padrao)`: Captura parâmetros passados na URL via Query String (ex: `/?busca=python&categoria=Web+Design`).

Se `busca` não existir na URL, assume `""`.

Se `categoria` não existir, assume `"Todas"`.

`.strip()`: Remove espaços em branco desnecessários antes e depois do texto digitado.

```
Python
            postagens_visiveis   =   filtrar_postagens(POSTAGENS,
termo_busca, categoria_atual)
```

Executa a função lógica passando os dados coletados da URL, obtendo a lista filtrada de posts que devem aparecer na tela.

```
Python
    return render_template(

        "index.html",

        postagens=postagens_visiveis,

        comentarios=COMENTARIOS,

        termo_busca=termo_busca,

        categoria_atual=categoria_atual,
```

```
        aviso_sucesso=None,
        erro_404=False
    )
```

Carrega o arquivo `index.html` e injeta nele as variáveis:

`postagens`: Apenas os posts filtrados.

`comentarios`: A lista com todos os comentários salvos.

`termo_busca` e `categoria_atual`: Para manter os campos do formulário preenchidos com a seleção atual do usuário.

`aviso_sucesso=None`: Indica ao HTML que nenhuma mensagem de confirmação precisa ser exibida no momento.

`erro_404=False`: Sinaliza que a página foi encontrada com sucesso.

5. Rota de Comentários (`/comentar`)

Python

```
@aplicacao.route("/comentar", methods=["POST"])
def adicionar_comentario():
```

Mapeia o caminho `/comentar` aceitando exclusivamente o método **POST**, usado no envio de dados de formulários HTML.

Python

```
    nome_aluno = request.form.get("nome", "").strip()
    texto_mensagem = request.form.get("mensagem", "").strip()
```

`request.form.get(...)`: Extrai os valores submetidos através dos campos `<input name="nome">` e `<textarea name="mensagem">` do formulário HTML.

```python
    if nome_aluno and texto_mensagem:
        novo_item = {
            "nome": nome_aluno,
            "mensagem": texto_mensagem
        }
        COMENTARIOS.insert(0, novo_item)
        mensagem_retorno = f"Comentário de {nome_aluno} salvo com sucesso!"
    else:
        mensagem_retorno = "Preencha seu nome e sua mensagem para publicar."
```

Valida se ambos os campos foram preenchidos (strings não vazias).

Se válidos: cria um novo dicionário e utiliza **`COMENTARIOS.insert(0, novo_item)`** para adicioná-lo na primeira posição (índice 0), garantindo que o comentário mais recente apareça no topo da lista. Define uma mensagem de sucesso com interpolação de string (`f"..."`).

Se algum estiver vazio: não insere na lista e define uma mensagem de erro/alerta.

```python
    return render_template(
        "index.html",
        postagens=POSTAGENS,
        comentarios=COMENTARIOS,
        termo_busca="",
        categoria_atual="Todas",
        aviso_sucesso=mensagem_retorno,
```

```
        erro_404=False
    )
```

Renderiza novamente o `index.html` exibindo a lista atualizada de comentários e o aviso gerado em `aviso_sucesso`.

6. Tratamento de Erro 404

Python

```
@aplicacao.errorhandler(404)
def pagina_nao_encontrada(erro):
```

`@aplicacao.errorhandler(404)`: Intercepta qualquer requisição a rotas que não existem na aplicação (ex: `/contato`, `/admin`).

O parâmetro `erro` recebe os detalhes do erro gerado pelo servidor.

Python

```
    return render_template(
        "index.html",
        postagens=POSTAGENS,
        comentarios=COMENTARIOS,
        termo_busca="",
        categoria_atual="Todas",
        aviso_sucesso=None,
        erro_404=True
    ), 404
```

Renderiza a página principal passando a flag `erro_404=True`, permitindo que o template HTML exiba um modal ou aviso visual de página não encontrada.

O valor `, 404` após o `render_template` define formalmente o código de status HTTP da resposta como **404 Not Found**, mantendo a semântica correta do protocolo web.

7. Inicialização do Servidor

```python
if __name__ == "__main__":

    aplicacao.run(debug=True, port=5000)
```

`if __name__ == "__main__":`: Garante que o servidor só iniciará se o arquivo for executado diretamente pelo terminal (`python app.py`), evitando execuções acidentais caso este arquivo seja importado como módulo por outro script.

`aplicacao.run(...)`: Inicia o servidor de desenvolvimento embutido do Flask:

> **`port=5000`**: Define que o site estará acessível no endereço `http://localhost:5000` (ou `[http://127.0.0.1:5000](http://127.0.0.1:5000)`).
>
> **`debug=True`**: Habilita o modo de depuração, que reinicia o servidor automaticamente sempre que você salvar alterações no código e exibe rastreamentos de erro detalhados no navegador se ocorrer uma falha.